МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №3

Тема: **«Функциональное тестирование»**

Дисциплина: **«Тестирование и отладка программного обеспечения»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: для разработанного приложения (пп. 1.4) продемонстрировать на выбор применение технологии функционального тестирования: «Интеграционное тестирование».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Анализ функциональных требований

На основе кода и архитектуры выделены следующие функциональные и нефункциональные требования к мультиплеерному режиму:

1) *Генерация уровня*: Уровень должен создаваться ровно для двух игроков (разных цветов) и содержать 11 монет (10 обычных по 10 очков и 1 бонусная на 20 очков).

2) *Контракты фабрики*: Фабрика должна строго соблюдать типы создаваемых объектов. Билдер обязан отклонять некорректные типы.

3) *Управление и физика*: Композит игроков должен обрабатывать кадры ввода для обоих игроков, перемещать их и удерживать в пределах границ уровня.

4) *Механика сбора*: Игроки собирают монеты при пересечении. Обычная монета = 10 очков, бонусная = 20 очков. При наложении обоих игроков на одну монету приоритет у Игрока 1.

5) *Состояние матча*: Матч завершается только при сборе всех монет. После завершения игра блокируется. Формируется корректное сообщение о результате (Победа Игрока 1 / Игрока 2 / Ничья).

2. Создание тестовых сценариев на основе функциональных требований

Разработаны следующие группы интеграционных тестовых сценариев:

1) Сборка уровня и валидация фабрики: Проверка количества игроков, типов монет, подсчёта вызовов фабрики и обработки ошибок при возврате неверных типов.

2) Валидация контрактов матча: Проверка ограничения на количество игроков (строго 2), обработка пустых списков и несовпадения кадров ввода.

3) Движение и границы: Сценарии одновременного движения обоих игроков и проверка их удержания внутри границ карты при непрерывном движении.

4) Сбор монет и начисление очков: Проверка сбора обычных и бонусных монет, накопления очков, приоритета Игрока 1 при коллизии.

5) Финализация матча: Проверка условия завершения (исчезновение всех монет), генерации сообщений о победе/ничьей и блокировки обновления состояния после финиша.

Ниже приведён полный листинг тестов.

```
"""Интеграционные тесты мультиплеера «Собиратель монет».

Проверяют совместную работу фабрики, билдера уровня, сущностей и
фасада
TwoPlayerMatch / композита PlayerComposite.
"""

from __future__ import annotations

from types import SimpleNamespace

import pytest

from coin_collector.entities import BonusCoin, Coin
from coin_collector.factory import ArcadeObjectFactory, ObjectKind
from coin_collector.level import GameLevelBuilder,
build_two_player_level
from coin_collector.multiplayer import PlayerComposite, TwoPlayerMatch


SIZE = (900, 700)
FRAME = 0.016


# ----------------------------------------------------------------------
------- #
# Вспомогательные утилиты
# ----------------------------------------------------------------------
------- #

def frame(horizontal: float = 0.0, vertical: float = 0.0) ->
```

```
SimpleNamespace:
    """Duck-typed замена InputFrame; match читает только .movement."""
    return SimpleNamespace(movement=(horizontal, vertical))


def build_level(player_positions, coin_positions=(),
bonus_positions=()):
    factory = ArcadeObjectFactory()
    builder = GameLevelBuilder(factory, SIZE)
    builder.set_player_start(player_positions[0])
    for position in player_positions[1:]:
        builder.add_player_start(position)
    for position in coin_positions:
        builder.add_coin(position)
    for position in bonus_positions:
        builder.add_bonus_coin(position)
    return builder.build()


def make_match(player_positions, coin_positions=(),
bonus_positions=()):
    return TwoPlayerMatch(build_level(player_positions,
coin_positions, bonus_positions))


def coins_near(center, count):
    offsets = [(0, 0), (0, 20), (0, -20)]
    return [(center[0] + dx, center[1] + dy) for dx, dy in
offsets[:count]]


# ------------------------------------------------------------------------
------- #
# Сборка уровня + фабрика
# ------------------------------------------------------------------------
------- #

def test_two_player_level_contains_two_players_and_a_bonus_coin():
    level = build_two_player_level(ArcadeObjectFactory(), SIZE)

    assert len(level.players) == 2
    assert level.players[0].color != level.players[1].color

    bonus = [coin for coin in level.coins if isinstance(coin,
BonusCoin)]
    assert len(bonus) == 1
    assert bonus[0].value == 20            # 10 * 2
    assert len(level.coins) == 11          # 10 обычных + 1 бонус


class TrackingFactory(ArcadeObjectFactory):
    def __init__(self):
        super().__init__()
        self.calls: list[ObjectKind] = []

    def create(self, kind, position):
        self.calls.append(kind)
        return super().create(kind, position)
```

```python
def test_two_player_level_goes_through_the_factory():
    factory = TrackingFactory()
    build_two_player_level(factory, SIZE)

    assert factory.calls.count(ObjectKind.PLAYER) == 2
    assert factory.calls.count(ObjectKind.COIN) == 11   # 10 + бонус
    assert factory.calls.count(ObjectKind.HAZARD) == 0


class BrokenFactory(ArcadeObjectFactory):
    def create(self, kind, position):
        if kind is ObjectKind.PLAYER:
            return Coin(position)           # неправильный тип
        return super().create(kind, position)


def test_builder_rejects_factory_returning_wrong_types():
    with pytest.raises(TypeError):
        build_two_player_level(BrokenFactory(), SIZE)


# ------------------------------------------------------------------------
------- #
# Контракты TwoPlayerMatch / PlayerComposite
# ------------------------------------------------------------------------
------- #

@pytest.mark.parametrize("player_count", [1, 3])
def test_match_requires_exactly_two_players(player_count):
    positions = [(200 + 100 * i, 300) for i in range(player_count)]
    level = build_level(positions)
    with pytest.raises(ValueError):
        TwoPlayerMatch(level)


def test_player_composite_rejects_empty_players():
    with pytest.raises(ValueError):
        PlayerComposite([])


def test_player_composite_rejects_mismatched_frames():
    level = build_two_player_level(ArcadeObjectFactory(), SIZE)
    composite = PlayerComposite(level.players)
    with pytest.raises(ValueError):
        composite.update((frame(),), FRAME, level.bounds)


def test_player_composite_moves_every_player():
    level = build_two_player_level(ArcadeObjectFactory(), SIZE)
    composite = PlayerComposite(level.players)
    start_one = composite.players[0].rect.topleft
    start_two = composite.players[1].rect.topleft

    composite.update((frame(1, 0), frame(0, 1)), 0.1, level.bounds)

    assert composite.players[0].rect.x > start_one[0]
```

```
    assert composite.players[1].rect.y > start_two[1]


# ------------------------------------------------------------------------
------- #
# Движение через фасад
# ------------------------------------------------------------------------
------- #

def test_match_moves_both_players():
    match = make_match([(300, 300), (700, 300)], coin_positions=[(500,
400)])
    p1, p2 = match.level.players
    start_one = p1.rect.topleft
    start_two = p2.rect.topleft

    match.update(frame(1, 0), frame(0, 1), 0.1)

    assert p1.rect.x > start_one[0] and p1.rect.y == start_one[1]
    assert p2.rect.y > start_two[1] and p2.rect.x == start_two[0]


def test_match_keeps_players_inside_bounds():
    match = make_match([(300, 300), (700, 300)], coin_positions=[(500,
500)])

    for _ in range(300):
        match.update(frame(-1, -1), frame(1, 1), FRAME)

    for player in match.level.players:
        assert match.level.bounds.contains(player.rect)


# ------------------------------------------------------------------------
------- #
# Сбор монет и подсчёт очков
# ------------------------------------------------------------------------
------- #

def test_player_collects_coin_and_gains_score():
    match = make_match([(300, 300), (700, 300)], coin_positions=[(300,
300)])

    collected = match.update(frame(), frame(), FRAME)

    assert collected == 1
    assert match.coin_counts == [1, 0]
    assert match.scores == [10, 0]
    assert match.level.coins == []


def test_bonus_coin_gives_double_points():
    match = make_match([(300, 300), (700, 300)],
bonus_positions=[(300, 300)])

    (bonus,) = match.level.coins
    assert isinstance(bonus, BonusCoin)
    assert bonus.value == 20
```

```
    collected = match.update(frame(), frame(), FRAME)

    assert collected == 1
    assert match.scores == [20, 0]
    assert match.coin_counts == [1, 0]


def test_player_one_has_priority_when_both_overlap_same_coin():
    match = make_match([(300, 300), (300, 300)], coin_positions=[(300,
300)])

    match.update(frame(), frame(), FRAME)

    assert match.coin_counts == [1, 0]
    assert match.scores == [10, 0]


def test_update_returns_number_of_collected_coins():
    match = make_match(
        [(300, 300), (700, 300)],
        coin_positions=[(300, 300), (300, 320)],
    )
    assert match.update(frame(), frame(), FRAME) == 2


def test_scores_accumulate_across_updates():
    match = make_match(
        [(300, 300), (700, 300)],
        coin_positions=[(300, 300), (400, 300)],
    )

    match.update(frame(), frame(), FRAME)
    assert match.scores == [10, 0]
    assert not match.finished

    for _ in range(200):
        match.update(frame(1, 0), frame(), FRAME)
        if match.finished:
            break

    assert match.finished
    assert match.coin_counts == [2, 0]
    assert match.scores == [20, 0]


# ---------------------------------------------------------------------
------- #
# Финализация матча и сообщение о результате
# ---------------------------------------------------------------------
------- #

def test_match_finishes_only_when_all_coins_are_gone():
    match = make_match(
        [(300, 300), (700, 300)],
        coin_positions=[(300, 300), (500, 300)],
    )
```

```
    match.update(frame(), frame(), FRAME)
    assert not match.finished
    assert len(match.level.coins) == 1

    for _ in range(200):
        match.update(frame(1, 0), frame(), FRAME)
        if match.finished:
            break
    assert match.finished


def test_result_message_is_available_only_after_finish():
    match = make_match([(300, 300), (700, 300)], coin_positions=[(300,
300)])

    with pytest.raises(RuntimeError):
        _ = match.result_message

    match.update(frame(), frame(), FRAME)
    assert match.result_message


@pytest.mark.parametrize(
    "p1_count, p2_count, expected",
    [
        (1, 1, "Ничья!"),
        (2, 1, "Игрок 1 победил!"),
        (1, 2, "Игрок 2 победил!"),
    ],
)
def test_result_message_announces_correct_outcome(p1_count, p2_count,
expected):
    coins = coins_near((300, 300), p1_count) + coins_near((700, 300),
p2_count)
    match = make_match([(300, 300), (700, 300)], coin_positions=coins)

    match.update(frame(), frame(), FRAME)

    assert match.finished
    assert match.coin_counts == [p1_count, p2_count]
    assert match.result_message == expected


def test_update_after_finish_is_a_noop():
    match = make_match([(300, 300), (700, 300)], coin_positions=[(300,
300)])
    match.update(frame(), frame(), FRAME)
    assert match.finished

    collected = match.update(frame(1, 0), frame(0, 1), 0.1)

    assert collected == 0
    assert match.scores == [10, 0]
    assert match.coin_counts == [1, 0]
```

3. Подготовка тестовых данных

Для изоляции тестов и проверки граничных случаев подготовлены следующие данные и заглушки:

1) Мок-объекты ввода: Использован SimpleNamespace для эмуляции InputFrame (передача векторов движения по осям X и Y).

2) Координатные сетки: Заранее подготовленные наборы координат для спавна игроков и монет (например, функция coins_near для генерации скоплений монет вокруг точки).

3) Тестовые фабрики: Созданы кастомные классы TrackingFactory (для подсчёта вызовов) и BrokenFactory (для намеренной генерации объектов неверного типа и проверки обработки исключений).

4. Выполнение тестов и сравнение фактических результатов с ожидаемыми

Тестирование проводилось с использованием фреймворка pytest. Все тесты, включая параметризованные, выполнены успешно.

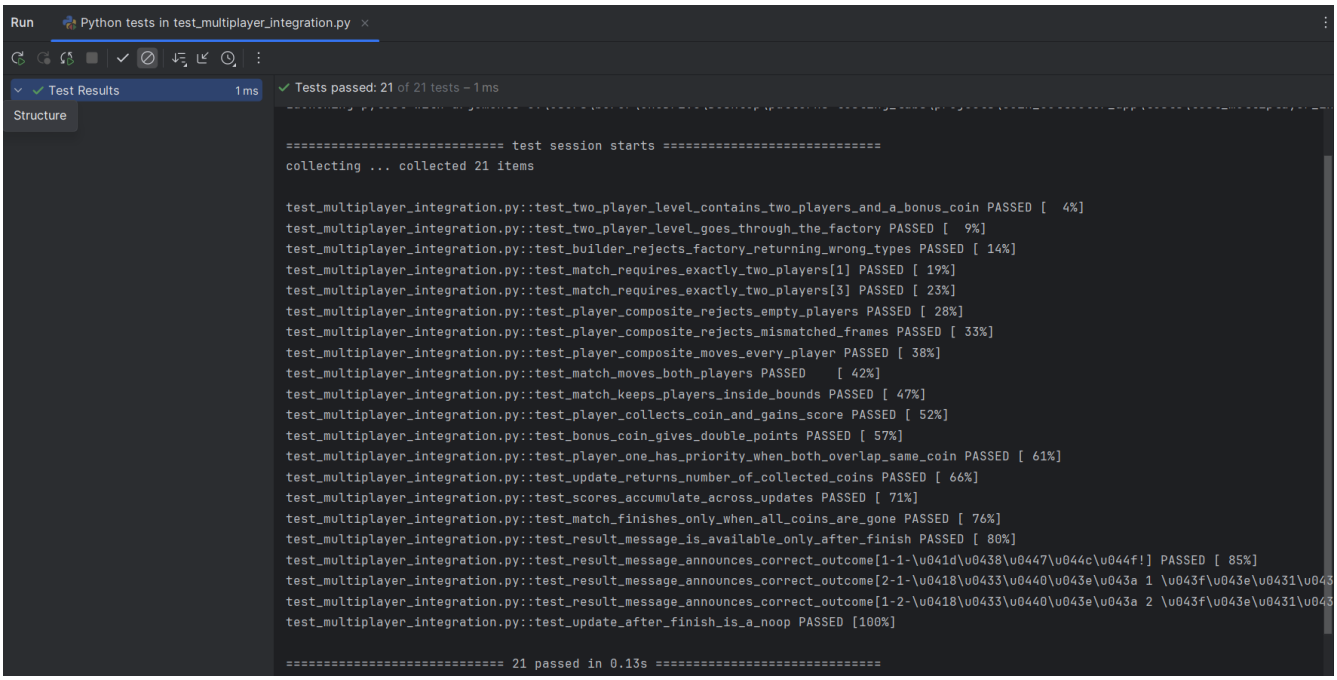


Рисунок 1. Запуск тестов в PyCharm

Сравнение результатов:

1) Фактическое количество игроков и монет на уровне совпало с ожидаемым (2 игрока, 11 монет).

2) Очки начисляются корректно (массивы scores и coin_counts формируются верно, например [10, 0] или [20, 0]).

3) Физическая модель работает стабильно: игроки не выходят за пределы level.bounds.

4) Логика финализации отрабатывает без сбоев: result_message генерируется только после match.finished, а последующие вызовы update() возвращают 0 собранных монет и не меняют счёт.

5. Документирование и отслеживание обнаруженных дефектов и повторное тестирование

- *Количество обнаруженных дефектов:* 0.

- *Критичность:* Отсутствует.

- *Повторное тестирование:* Не требуется.

Заключение: Интеграция компонентов мультиплеера (PlayerComposite, TwoPlayerMatch, GameLevelBuilder) реализована корректно. Дефектов, regressions или нарушений контрактов между модулями не выявлено. Код готов к дальнейшему развитию или передаче в релиз.

Вывод:

Проделав работу, применил технологию интеграционного тестирования для разработанного приложения.